22. Sorting and Searching

22. Sorting and Searching

Phase: Phase 4 — Algorithms and data structures
Estimated time: 1–2 days
Slug: `22-sorting-searching`

Learning objectives

- Use sort package effectively
- Implement binary search correctly
- Know comparison contracts
- Avoid off-by-one errors

Prerequisites

Complete prior numbered lessons. Type every example yourself.

sort package

```
sort.Ints(xs)
sort.Slice(people, func(i, j int) bool { return people[i].Age < people[j].Age })
```

Binary search

Requires sorted input. Classic bug: midpoint overflow (use `lo+(hi-lo)/2`).

```
func Search(xs []int, target int) int {
    lo, hi := 0, len(xs)
    for lo < hi {
        mid := lo + (hi-lo)/2
        if xs[mid] < target { lo = mid + 1 } else { hi = mid }
    }
    if lo < len(xs) && xs[lo] == target { return lo }
    return -1
}
```

Stability

`sort.SliceStable` preserves equal-element order — useful for multi-key sorts.

Common mistakes (especially from Python instincts)

- Binary search on unsorted data
- Inclusive vs exclusive bounds confusion
- Expensive comparisons in comparators

Exercises

Work in `exercises/22-sorting-searching/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.

- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/22-sorting-searching/` until you have an honest failing attempt.

Self-check

1. Preconditions for binary search?
2. When use SliceStable?
3. What is mid overflow?

Further reading

- https://pkg.go.dev/sort